

Linux应用

8.2 Linux系统调用的实现

核心思想：用户空间与内核空间

现代操作系统为了保护系统稳定与安全，将虚拟内存空间划分为两部分：

- **用户空间 (User Space)**：普通应用程序运行的地方，权限受限，无法直接访问硬件和内核内存。
- **内核空间 (Kernel Space)**：操作系统内核运行的地方，拥有最高权限，可以访问所有硬件和内存。

触发机制：从用户态到内核态

当应用程序需要内核服务时，它会通过标准库（如 glibc）提供的封装函数（如 `printf` 内部调用 `write`）发起请求。这个封装函数会执行一个特殊的 CPU 指令来触发系统调用。

- **传统方式 (软中断)**：在 x86 架构中，主要通过 `int 0x80` 软中断指令实现。程序将系统调用号和参数放入特定寄存器，执行 `int 0x80` 后，CPU 会暂停当前任务，切换到内核态，并跳转到预设的中断处理程序。
- **现代方式 (专用指令)**：为了提高性能，现代 64 位系统（如 x86-64）普遍采用更高效的 `syscall` 指令。其核心思想与软中断类似，但切换速度更快。

8.6 匿名管道 (pipe) 与命名管道(FIFO)

- **半双工(Half-Duplex)**：指通信双方可以交替进行双向数据传输，但同一时刻只能单向传输。
- **全双工(Full-Duplex)**：通信双方可以同时同时进行双向数据传输。
- **单工(Simplex)**：数据只能单向传输，无法实现双向通信。

8.6.1 pipe

- 半双工
- 生命周期：依附于创建它的进程
- 匿名：只能在父子、兄弟进程间单向通信
- 创建：

```
1 #include <unistd.h> // 包含 pid_t 和 fork() 的定义
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main() {
6     pid_t pid; // 1. 声明变量，准备接收结果
```

`pid_t pid;` 这行代码是在声明一个变量，专门用来存储“进程ID”。

1. pid_t：数据类型

- 含义：它是 "Process ID Type" 的缩写。

2. pid：变量名

- 作用：接收 fork() 函数的返回值。

pid 的值	含义	你的身份
> 0 (正整数)	返回的是子进程的 ID	你是父进程
== 0	返回 0	你是子进程
< 0	创建失败	发生错误

```
1 // 1. 创建管道
2 int pipefd[2]; // fd是"文件描述符" (File Descriptor) 的缩写
3 int check = pipe(pipefd);
```

核心功能：创建管道

pipe(pipefd) 是 Linux 系统调用，用于创建一个匿名管道。

- 输入/输出：它接收一个包含两个整数的数组 pipefd 作为参数。
 - pipefd[0]：读端文件描述符。用于从管道中读取数据。
 - pipefd[1]：写端文件描述符。用于向管道中写入数据。
- 数据流向：数据是单向流动的，从写端 pipefd[1] 流入，从读端 pipefd[0] 流出。

健壮性设计：错误处理

```
1 if (check == -1)
2 {
3     perror("pipe");
4     exit(EXIT_FAILURE);
5 }
```

- 检查返回值：pipe() 函数在成功时返回 0，失败时返回 -1。
- perror("pipe")：当创建失败时（例如系统文件描述符耗尽），这行代码会打印出详细的错误原因。它会输出你传入的字符串 "pipe"，后跟一个冒号和具体的系统错误信息（如 "Too many open files"），这对于调试非常有帮助。
- exit(EXIT_FAILURE)：立即终止当前程序的运行，并向操作系统报告“程序出错了”或“执行失败”。

`exit()`: 终止程序的函数

`EXIT_FAILURE`: 代表“失败”的常量

```
1 // 2. 创建子进程
2 pid = fork();
3 if (pid == -1) {
4     perror("fork");
5     exit(EXIT_FAILURE);
6 }
7
8 if (pid == 0) {
9     // --- 子进程代码 ---
10    close(pipefd[1]); // 关闭不需要的写端
11
12    char buffer[100];
13    read(pipefd[0], buffer, sizeof(buffer)); // 从读端读取
14    printf("子进程收到: %s\n", buffer);
15
16    close(pipefd[0]); // 通信结束, 关闭读端
17 } else {
18
```

- `pid = fork()`; 创建子进程
- `if (pid == -1)` 子进程错误处理
- `close(pipefd[i])`; 关闭不必要的端
- `read(pipefd[0], buffer, sizeof(buffer))`; 从读端读取, 存到buffer数组, 读取大小为 `sizeof(buffer)` 的数据。

参数位置	代码片段	含义与作用
第1个参数	<code>pipefd[0]</code>	指定“从哪读”。这是管道的读端文件描述符。在之前的 <code>pipe(pipefd)</code> 调用中, 系统已经规定 <code>pipefd[0]</code> 专门用于读取数据。
第2个参数	<code>buffer</code>	指定“存到哪”。这是一个预先定义好的字符数组 (如 <code>char buffer[100]</code>)。读取到的数据会被原封不动地填进这个内存区域。
第3个参数	<code>sizeof(buffer)</code>	指定“读多少”。告诉系统缓冲区有多大 (例如 100 字节), 防止读取的数据太多导致溢出 (覆盖其他内存)。

```
1 // --- 父进程代码 ---
2     close(pipefd[0]); // 关闭不需要的读端
3
4     const char *msg = "Hello, Child!";
5     write(pipefd[1], msg, strlen(msg) + 1); // 向写端写入
6
7     close(pipefd[1]); // 通信结束，关闭写端
8 }
9
10 return 0;
11 }
```

- `write(pipefd[1], msg, strlen(msg) + 1);` 像写端写入，数据源为msg，写入的数据大小为 `strlen(msg) + 1`。

参数位置	代码片段	含义与作用
第1个参数	<code>pipefd[1]</code>	指定“往哪写”。这是管道的写端文件描述符。在之前的 <code>pipe()</code> 调用中，系统规定 <code>pipefd[1]</code> 专门用于写入数据。
第2个参数	<code>msg</code>	指定“写什么”。这是要发送的数据源，通常是一个字符串或字符数组（例如 <code>const char *msg = "Hello";</code> ）。
第3个参数	<code>strlen(msg) + 1</code>	指定“写多少”。这是最关键的一点。 <code>strlen(msg)</code> 计算字符串的有效长度（不包含结尾的空字符 <code>\0</code> ）。 <code>+ 1</code> 是为了把字符串末尾的 <code>\0</code> 也一起写入管道。

8.6.2 FIFO命名管道

- 半双工
- 生命周期：独立于进程存在，手动结束
- 命名：与任何已知名字的进程通信
- 创建：

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <sys/stat.h>
4 #include <errno.h>
5
6 int main() {
7     const char *fifo_path = "/tmp/my_fifo";
8
9     // 尝试创建命名管道，权限设置为 0666（所有用户可读写）
10    if (mkfifo(fifo_path, 0666) == -1) {
11        // 如果管道已存在 (errno == EEXIST)，通常不算错误，可以直接使用
12        if (errno != EEXIST) {
13            perror("mkfifo 失败");
14            return 1;
15        }
16        printf("管道已存在，直接使用。\\n");
17    } else {
18        printf("命名管道创建成功: %s\\n", fifo_path);
19    }
20
21    // 后续可以使用 open(), read(), write() 操作该管道
22    // ...
23
24    return 0;
25 }
```

八进制值	符号表示	含义	适用场景
0666	rw-rw-rw-	所有人均可读、写	最常用，允许任意进程间通信
0644	rw-r--r--	所有者可读写，其他人只读	类似普通文本文件权限
0600	rw-----	仅所有者可读写	高安全性，私有通信
0777	rwxrwxrwx	所有人可读、写、执行	极少用于管道（执行位对管道无意义）

8.10 I/O多路复用机制

核心定义：I/O多路复用让单个进程/线程能够同时监控多个文件描述符（socket、文件、管道等），当其中任意一个就绪时，程序就能立即知道并处理它。

8.10.1 select：最基础的"监听器"

一、核心概念：位图（Bitmap）

`fd_set` 本质上是一个**位数组**（bitmap），每个 bit 代表一个文件描述符（fd）：

1	位数组：	[0]	[0]	[1]	[0]	[0]	[1]	[0]	...	[0]
2	索引：	0	1	2	3	4	5	6		1023
3		↑			↑		↑			
4			不监视		监视	fd=2		↑		
5							监视	fd=5		

- 第 n 位为 **1** = 监视 fd = n 的 socket
- 第 n 位为 **0** = 不监视

二、四个配套宏（必须一起用）

宏	作用	类比
<code>FD_ZERO(&set)</code>	清空整个集合（所有位变0）	把小本本擦干净
<code>FD_SET(fd, &set)</code>	添加 fd 到集合	监视这个fd
<code>FD_CLR(fd, &set)</code>	移除 fd 从集合（对应位变0）	不监视这个fd
<code>FD_ISSET(fd, &set)</code>	检查 fd 是否在集合中（位是否为1）	看这个fd来没来数据

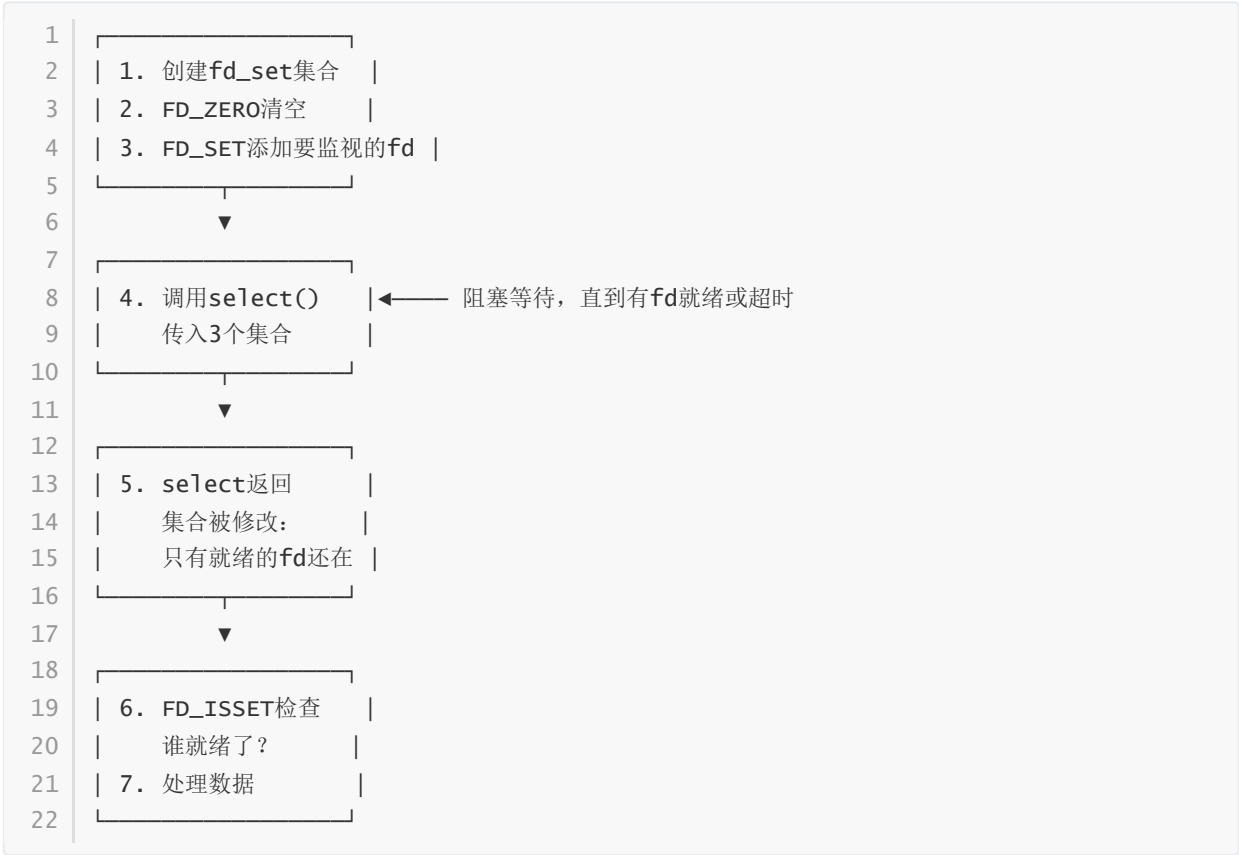
三、函数原型

1	<code>#include <sys/select.h></code>	
2		
3	<code>int select(int nfds,</code>	<code>// 最大fd值 + 1</code>
4	<code>fd_set *readfds,</code>	<code>// 监视"可读"的集合（传入传出）（可NULL）</code>
5	<code>fd_set *writefds,</code>	<code>// 监视"可写"的集合（可NULL）</code>
6	<code>fd_set *exceptfds,</code>	<code>// 监视"异常"的集合（可NULL）</code>
7	<code>struct timeval *timeout);</code>	<code>// 超时时间（NULL表示阻塞）</code>

返回值：

- `> 0`：有几个 fd 就绪了就返回几
- `0`：超时了，没 fd 就绪
- `-1`：出错

四、核心用法步骤



五、关键点

要点	说明
nfds = max_fd + 1	不是fd数量，是最大值+1。比如监视fd=3和fd=5，传6
集合是"传入传出"	select会修改你的集合，把没就绪的fd清掉
每次循环重新FD_SET	因为集合被修改了，下次select前要重新添加
不能传NULL给所有集合	至少有一个集合非NULL，否则select直接返回错误
timeout = NULL	永久阻塞，直到有fd就绪
timeout = {0, 0}	非阻塞，立即返回，用于轮询

struct timeval *timeout参数:

```
1 NULL//永久阻塞，直到有数据来了
2 struct timeval timeout = {0, 0}; //立即返回（非阻塞）--看一眼有没有数据来
3 struct timeval timeout = {5, 0}; //具体超时时间{秒，微妙}--等具体的时间看有没有数据来
4 int n = select(max_fd + 1, &set, NULL, NULL, timeout);
```

完整处理模板

假如监视3个fd是否可读:

```
1  int client[3] = {4, 5, 6}; // 3个socket
2
3  // 构造监视集合
4  fd_set readfds;
5  FD_ZERO(&readfds);
6  for (int i = 0; i < 3; i++) {
7      FD_SET(client[i], &readfds);
8  }
9
10 // 等待
11 int ret = select(7, &readfds, NULL, NULL, NULL);
12
13 // select 返回 2 (假设fd4和fd6同时来数据)
14
15 // 检查谁举手了
16 for (int i = 0; i < 3; i++) {
17     int fd = client[i];
18
19     if (FD_ISSET(fd, &readfds)) {
20         // 只有 fd=4 和 fd=6 会进这里, fd=5 不会
21         char buf[1024];
22         int n = read(fd, buf, sizeof(buf));
23         printf("从 fd=%d 读到 %d 字节\n", fd, n);
24     }
25 }
```